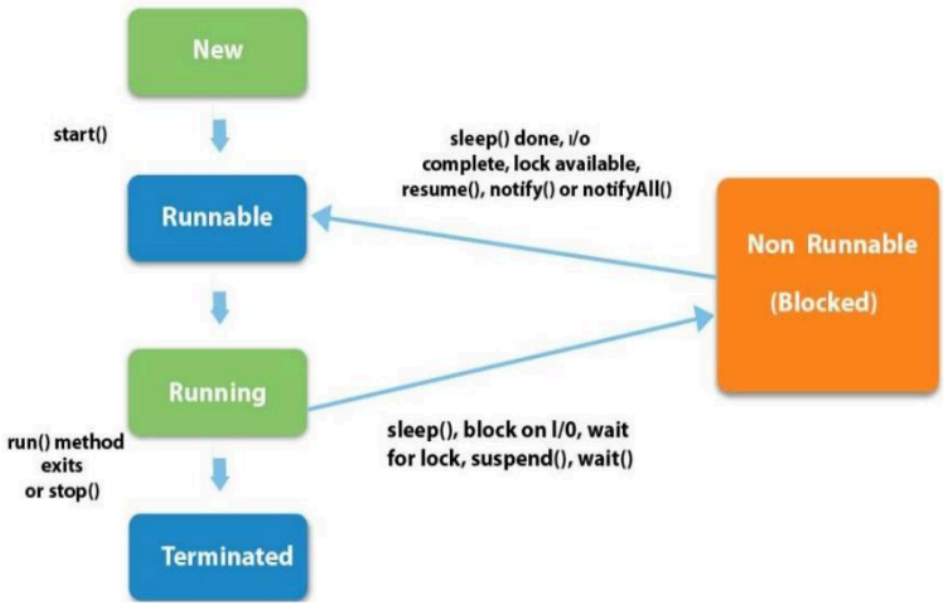
Unit -3

Chapter 1- Multithreaded Programming

1.Explain the lifecycle of a thread in java with diagram

The **life cycle of a thread** represents the different states a thread goes through from its creation to termination. According to your uploaded notes (Multithreading unit), this lifecycle is controlled by the **JVM**.



🔄 Thread States Explained

From your material (*see diagram on page 6 of Multithreaded Programming*), the thread passes through **5 main states**:

1️⃣ New (Born State)

- Thread object is created using new Thread()
- It is not yet started (no execution)

Example:
Thread t = new Thread();

-

2️⃣ Runnable (Ready State)

- Thread moves to this state after calling start()
- It is ready to run but waiting for CPU scheduling
- JVM thread scheduler decides when it runs

3️⃣ Running (Execution State)

- Thread is actively executing its run() method
- This is where actual task execution happens

4️⃣ Non-Runnable / Blocked State

- Thread is temporarily inactive
- It enters this state due to:
  - sleep()
  - wait()
  - Waiting for I/O
  - Waiting for a lock
- Once the condition is satisfied, it goes back to **Runnable**

5️⃣ Terminated (Dead State)

- Thread completes execution of run() method
- Or it is stopped
- It cannot be restarted again

🔁 State Transitions (Important for Exam)

- New → Runnable → when start() is called
- Runnable → Running → when CPU is assigned
- Running → Blocked → due to wait/sleep/I/O
- Blocked → Runnable → after condition completion
- Running → Terminated → when execution ends

💡 Key Point (Very Important)

- Calling run() directly **does NOT start a new thread**
- Only start() creates a new thread and moves it to Runnable state

---

✍️ Short Conclusion (for writing in exam)

The thread lifecycle in Java consists of five states: **New, Runnable, Running, Blocked, and Terminated**. A thread is created in the New state, becomes Runnable after start(), executes in Running state, may move to Blocked due to waiting conditions, and finally ends in Terminated state after completing execution.

2.Explain Different ways to create a thread in java with Example.

1️⃣By Extending Thread Class

- Create a class that extends Thread
- Override the run() method
- Call start() to begin execution

✅ Example:

```
class MyThread extends Thread {
  public void run() {
    System.out.println("Thread is running...");
  }

  public static void main(String[] args) {
    MyThread t = new MyThread();
    t.start();   // starts thread
  }
}
```

✔️ Key Point:

- Here, your class **is itself a thread**
- Not suitable if your class already extends another class (Java doesn't support multiple inheritance)

2️⃣By Implementing Runnable Interface

- Create a class that implements Runnable
- Implement the run() method
- Pass object to Thread and call start()

✅ Example:

```
class MyThread implements Runnable {
  public void run() {
    System.out.println("Thread is running...");
  }

  public static void main(String[] args) {
    MyThread obj = new MyThread();
    Thread t = new Thread(obj);
    t.start();
  }
}
```

✔️ Key Point:

- Preferred method in real applications
- Allows class to extend another class

🔍 Important Notes

- run() → contains thread logic
- start() → actually creates a new thread and calls run() internally

---

✍️ Conclusion (Exam Ready)

Threads in Java can be created either by **extending the Thread class** or by **implementing the Runnable interface**. The Runnable approach is more flexible and commonly used.

3.Explain Synchronized method and Synchronized block with example and differentiate Synchronized method and synchronized block.

🔹 1. Synchronized Method

- Entire method is locked
- Only one thread can execute the method at a time
- Lock is applied on the **object (this)**

✅ Example:

```
class Table {
  synchronized void printTable(int n) {
    for(int i = 1; i <= 5; i++) {
      System.out.println(n * i);
    }
  }
}

class MyThread1 extends Thread {
  Table t;
  MyThread1(Table t) { this.t = t; }
  public void run() {
    t.printTable(5);
  }
}

class MyThread2 extends Thread {
  Table t;
  MyThread2(Table t) { this.t = t; }
  public void run() {
    t.printTable(10);
  }
}
```

🔹 2. Synchronized Block

- Only a specific block of code is locked
- More efficient (better performance)
- Lock can be applied to **any object**

✅ Example:

```
class Table {
  void printTable(int n) {
    synchronized(this) {
      for(int i = 1; i <= 5; i++) {
        System.out.println(n * i);
      }
```

```
    }
  }
}
```

🔍 Difference Between Synchronized Method and Block

Feature	Synchronized Method	Synchronized Block
Lock Scope	Entire method	Only part of method
Performance	Slower (locks whole method)	Faster (locks only needed code)
Lock Object	Implicit (this)	Explicit (any object)
Flexibility	Less flexible	More flexible

✍️ Conclusion (Exam Ready)

A synchronized method locks the entire method, while a synchronized block locks only a specific portion of code. Synchronized blocks are more efficient and flexible compared to synchronized methods.

4.Explain Interthread Communication in java.

Interthread Communication in Java

Interthread communication is a mechanism that allows multiple threads to **communicate and coordinate with each other**. It is mainly used when threads share resources and need to work in proper sequence (avoid busy waiting).

🔹 Why Interthread Communication?

- Avoids **CPU wastage** (no continuous checking)
- Ensures **proper coordination** between threads
- Prevents **race conditions and deadlocks**

🔹 Methods Used

Java provides three important methods (from Object class):

1. **wait()**
    - Causes current thread to release lock and go into waiting state
2. **notify()**
    - Wakes up one waiting thread
3. **notifyAll()**
    - Wakes up all waiting threads

These methods must be used inside a **synchronized block/method**

🔹 Example (Producer–Consumer)

```
class Shared {
  int data;
  boolean available = false;

  synchronized void produce(int value) {
    while(available) {
      try { wait(); } catch(Exception e) {}
    }
    data = value;
    System.out.println("Produced: " + value);
    available = true;
    notify();
  }

  synchronized void consume() {
    while(!available) {
      try { wait(); } catch(Exception e) {}
    }
    System.out.println("Consumed: " + data);
    available = false;
    notify();
  }
}
```

🔄 Working

- Producer produces data → calls notify()
- Consumer waits using wait() until data is available
- Threads coordinate without wasting CPU

✍️ Conclusion (Exam Ready)

Interthread communication in Java is achieved using wait(), notify(), and notifyAll() methods. It allows threads to communicate efficiently by coordinating access to shared resources, avoiding unnecessary CPU usage.

Chapter 2- Collections

5.Explain the List interface and its methods.

List Interface in Java

The **List interface** is part of the Java Collection Framework and represents an **ordered collection of elements**. It allows **duplicate elements** and maintains **insertion order**. It is available in the java.util package.

📊 List Interface Concept

🔹 Features of List Interface

- Maintains **insertion order**
- Allows **duplicate values**
- Supports **index-based access**
- Can store **null elements**
- Implemented by classes like:
  - ArrayList
  - LinkedList
  - Vector
  - Stack

🔹 Declaration Example

```
List<String> list = new ArrayList<>();
```

🔹 Common Methods of List Interface

Method	Description
add(E e)	Adds element to the list
add(int index, E e)	Adds element at specific position
get(int index)	Returns element at index
set(int index, E e)	Replaces element at index
remove(int index)	Removes element from index
remove(Object o)	Removes specific element
size()	Returns number of elements
isEmpty()	Checks if list is empty
contains(Object o)	Checks if element exists
clear()	Removes all elements

🔹 Example

```
import java.util.*;

class Demo {
   public static void main(String[] args) {
      List<String> list = new ArrayList<>();
      list.add("Apple");
      list.add("Banana");
      list.add("Apple");  // duplicate allowed

      System.out.println(list.get(1)); // Banana
   }
}
```

✍️ Conclusion (Exam Ready)

The List interface is used to store ordered collections that allow duplicates. It provides various methods to add, remove, access, and manipulate elements efficiently.

6.Explain ArrayList class with features and methods.

ArrayList Class in Java

The **ArrayList class** is a part of the Java Collection Framework and implements the **List interface**. It uses a **dynamic array** to store elements and allows **duplicate values** while maintaining **insertion order**.

📊 ArrayList Concept

🔹 Features of ArrayList

- Uses **dynamic array** (size grows automatically)
- Maintains **insertion order**
- Allows **duplicate elements**
- Provides **fast random access** (index-based)
- Not synchronized (not thread-safe by default)
- Can store **heterogeneous elements** (if not using generics)

🔹 Declaration Example

```
import java.util.*;

ArrayList<String> list = new ArrayList<>();
```

🔹 Common Methods of ArrayList

Method	Description
add(E e)	Adds element to list
add(int index, E e)	Inserts element at position
get(int index)	Returns element at index
set(int index, E e)	Updates element
remove(int index)	Removes element by index
remove(Object o)	Removes specific element
size()	Returns number of elements

isEmpty()	Checks if list is empty
contains(Object o)	Checks presence of element
clear()	Removes all elements

🔹 Example

```
import java.util.*;

class Demo {
  public static void main(String[] args) {
    ArrayList<Integer> list = new ArrayList<>();

    list.add(10);
    list.add(20);
    list.add(10); // duplicate allowed

    System.out.println(list);      // [10, 20, 10]
    System.out.println(list.get(1)); // 20
  }
}
```

✍️ Conclusion (Exam Ready)

ArrayList is a resizable array implementation of the List interface that allows duplicates, maintains order, and provides fast access to elements. It is widely used due to its flexibility and ease of use.

7. Explain LinkedList class and compare it with ArrayList.

LinkedList Class in Java

The **LinkedList class** is a part of the Java Collection Framework and implements the **List interface**. It internally uses a **doubly linked list** to store elements, where each element (node) contains data and links to the previous and next nodes.

📊 LinkedList Concept

unstop

Difference Between ArrayList and LinkedList

ArrayList	LinkedList
Uses a dynamic array to store its elements	Uses a doubly-linked list, where each element is a node
Provides fast random access (O(1)) to elements using an index	Requires sequential traversal to find an element, making access slow (O(n))
Insertion/deletion in the middle of the list is slow (O(n))	Insertion and deletion at any position are fast (O(1))
Lower memory overhead per element, as it only stores the data	Higher memory as each node stores not only the data but also two pointers

---

🔹 Features of LinkedList

- Uses **doubly linked list** structure
- Maintains **insertion order**
- Allows **duplicate elements**
- Efficient for **insertion and deletion**
- Not synchronized
- Does **not provide fast random access** (no direct indexing)

🔹 Declaration Example

```
import java.util.*;

LinkedList<String> list = new LinkedList<>();
```

🔹 Example

```
import java.util.*;

class Demo {
  public static void main(String[] args) {
    LinkedList<String> list = new LinkedList<>();
```

```
        list.add("A");
        list.add("B");
        list.add("C");

        list.addFirst("Start");
        list.addLast("End");

        System.out.println(list);
    }
}
```

🔍 Difference Between ArrayList and LinkedList

Feature	ArrayList	LinkedList
Data Structure	Dynamic Array	Doubly Linked List
Access Speed	Fast (index-based)	Slow (sequential access)
Insertion/Deletion	Slow (shifting needed)	Fast (no shifting)
Memory Usage	Less	More (extra pointers)
Use Case	Frequent access	Frequent insertion/deletion

✍️ Conclusion (Exam Ready)

LinkedList is a doubly linked list implementation of the List interface that allows efficient insertion and deletion operations. Compared to ArrayList, it is slower for accessing elements but better for dynamic data manipulation.

8. Explain Set interface and its operation in detail with example.

Set Interface in Java

The **Set interface** is a part of the Java Collection Framework and represents a collection of **unique elements** (no duplicates allowed). It is available in the java.util package and extends the Collection interface.

📊 Set Interface Concept

🔹 Features of Set Interface

- Does **not allow duplicate elements**
- Stores elements in **unordered manner** (in general)
- Allows **null element** (only one in most implementations like HashSet)
- Useful for **removing duplicates**
- Implemented by classes:
  - HashSet
  - LinkedHashSet
  - TreeSet

🔹 Common Operations (Methods)

Method	Description
add(E e)	Adds element (ignored if duplicate)
remove(Object o)	Removes element
contains(Object o)	Checks presence
size()	Returns number of elements
isEmpty()	Checks if set is empty
clear()	Removes all elements
iterator()	Used to traverse elements

🔹 Example

```
import java.util.*;

class Demo {
   public static void main(String[] args) {
      Set<Integer> set = new HashSet<>();
```

```
        set.add(10);
        set.add(20);
        set.add(10); // duplicate ignored

        System.out.println(set); // [10, 20] (order not guaranteed)

        System.out.println(set.contains(20)); // true
        set.remove(10);
        System.out.println(set);
    }
}
```

🔹 Types of Set

- **HashSet** → Unordered, fastest
- **LinkedHashSet** → Maintains insertion order
- **TreeSet** → Sorted (ascending order)

✍️ Conclusion (Exam Ready)

The Set interface is used to store unique elements without duplicates. It provides operations like add, remove, and search, and is commonly implemented using HashSet, LinkedHashSet, and TreeSet.

Chapter -3 Applets

9.Explain Java Applets and its features.

Java Applets and Its Features

A **Java Applet** is a small Java program that is **embedded in a web page** and executed inside a web browser or applet viewer. It runs on the **client side** and is mainly used to create dynamic and interactive web content.

🔹 Features of Java Applets

- **Client-side execution** → Runs in the user's browser
- **Platform independent** → Works on any system with JVM
- **Dynamic content** → Can create interactive GUI applications
- **Secure environment** → Runs in a restricted sandbox
- **No main() method** → Execution controlled by browser/JVM
- **Uses AWT/Swing** → For graphical user interface
- **Event-driven** → Responds to user actions like clicks, typing

---

🔹 Basic Structure of an Applet

```
import java.applet.Applet;
import java.awt.Graphics;

public class MyApplet extends Applet {
   public void paint(Graphics g) {
      g.drawString("Hello Applet", 50, 50);
   }
}
```

🔹 Life Cycle Methods

- init() → Called once when applet is initialized
- start() → Called when applet starts or restarts
- stop() → Called when applet is stopped
- destroy() → Called before applet is removed
- paint() → Used for drawing output

🔹 Advantages

- Fast execution (client-side)
- Secure execution environment
- Rich GUI support

🔹 Disadvantages

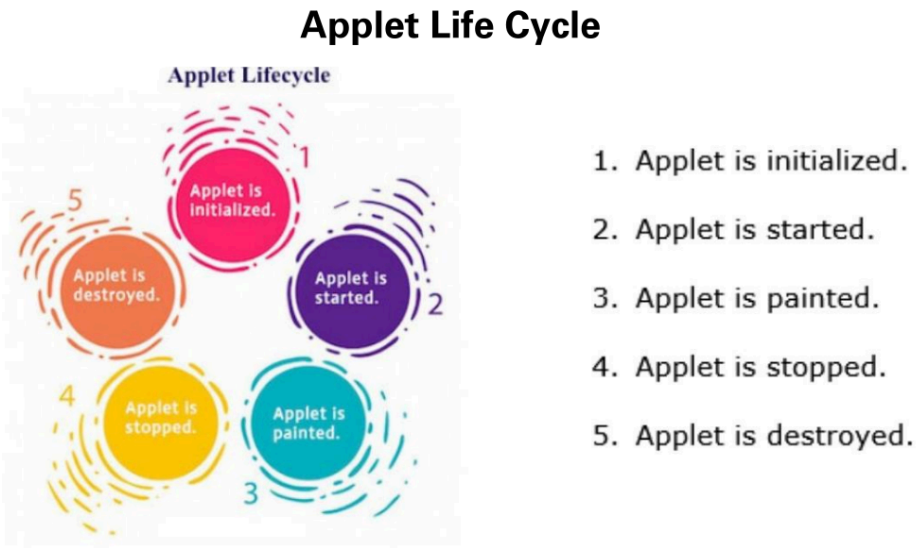
- Requires browser plugin
- Limited access to system resources

- Mostly **obsolete in modern web development**

✍️ Conclusion (Exam Ready)

Java Applets are small client-side programs embedded in web pages to create dynamic content. They are platform-independent, secure, and GUI-based, but are now outdated due to modern web technologies.

10. Explain Applet lifecycle methods with diagram.



Applet Life Cycle Methods in Java

The **Applet life cycle** defines the sequence of methods that are automatically invoked by the **JVM/browser** to control the execution of an applet. (*Refer to the lifecycle diagram in your notes – Event Handling unit*).

🔄 Life Cycle Methods

1️⃣ init()

- Called **only once** when the applet is loaded
- Used for initialization (variables, UI setup)

2️⃣ start()

- Called after init()
- Also called every time the applet becomes active again
- Used to start or resume execution

3️⃣ paint(Graphics g)

- Used to **display output** on the screen
- Called whenever the applet needs to redraw itself

4️⃣ stop()

- Called when the applet becomes inactive
- Example: when user switches to another page

5️⃣ destroy()

- Called when the applet is removed permanently
- Used to release resources

🔁 Flow of Execution

```
init() → start() → paint()
          ↓
        stop()
          ↓
   (start() again if needed)
          ↓
        destroy()
```

🔹 Example

```
import java.applet.Applet;
import java.awt.Graphics;

public class LifeCycleDemo extends Applet {
```

```
    public void init() {
        System.out.println("Applet Initialized");
    }

    public void start() {
        System.out.println("Applet Started");
    }

    public void paint(Graphics g) {
        g.drawString("Applet Running", 50, 50);
    }

    public void stop() {
        System.out.println("Applet Stopped");
    }

    public void destroy() {
        System.out.println("Applet Destroyed");
    }
}
```

✍️ Conclusion (Exam Ready)

The applet life cycle consists of five main methods: init(), start(), paint(), stop(), and destroy(). These methods control initialization, execution, rendering, suspension, and termination of an applet.

11. Explain getDocumentBase and getCodeBase methods.

getDocumentBase() and getCodeBase() Methods in Java Applet

Both methods are provided by the **Applet class** and are used to obtain **URL information** related to an applet. They help in locating files or resources on the web.

(*Concept based on Applet basics in your Event Handling unit*).

🔹 1. getDocumentBase()

- Returns the **URL of the HTML document** in which the applet is embedded
- Useful to locate resources relative to the **web page location**

✅ **Syntax:**

URL url = getDocumentBase();

✔️ **Example:**

If HTML file is:

http://example.com/folder/page.html

Then:

getDocumentBase() → http://example.com/folder/

🔹 2. getCodeBase()

- Returns the **URL of the applet class file (.class)**
- Useful to load resources relative to the **applet location**

✅ **Syntax:**

URL url = getCodeBase();

✔️ **Example:**

If applet class is:

http://example.com/classes/MyApplet.class

Then:

getCodeBase() → http://example.com/classes/

🔍 Difference Between getDocumentBase() and getCodeBase()

Feature	getDocumentBase()	getCodeBase()
Returns	URL of HTML page	URL of applet class
Based On	Document location	Applet location

Usage	Load files near webpage	Load files near applet
Example	page.html folder	.class file folder

🔹 Example Program

```
import java.applet.Applet;
import java.awt.Graphics;
import java.net.URL;

public class BaseDemo extends Applet {
  public void paint(Graphics g) {
    URL doc = getDocumentBase();
    URL code = getCodeBase();

    g.drawString("Document Base: " + doc, 20, 20);
    g.drawString("Code Base: " + code, 20, 40);
  }
}
```

✍️ Conclusion (Exam Ready)

getDocumentBase() returns the URL of the HTML document containing the applet, while getCodeBase() returns the URL of the applet class file. Both are used to locate resources in web-based applications.

Unit-4

Chapter 1- Networking basics

1.Discuss the Java Inet Address class with its methods and Applications.

Java InetAddress Class

The **InetAddress class** (from java.net package) is used to represent an **IP address** of a host (either IPv4 or IPv6). It helps in identifying machines on a network and performing basic networking operations like hostname resolution.

(*Covered in your Networking unit – Java InetAddress class section*).

🔹 Key Features

- Represents **IP address and host name**
- Supports both **IPv4 and IPv6**
- Used for **DNS lookup (hostname ↔ IP)**
- Cannot be instantiated directly (uses factory methods)

🔹 Important Methods

Method	Description
getLocalHost()	Returns local host address
getByName(String host)	Returns IP for given hostname
getAllByName(String host)	Returns all IPs for a host
getHostName()	Returns host name
getHostAddress()	Returns IP address as string
isReachable(int timeout)	Checks if host is reachable
getLoopbackAddress()	Returns loopback address (127.0.0.1)

🔹 Example Program

```
import java.net.*;

class Demo {
  public static void main(String[] args) {
    try {
      InetAddress ip = InetAddress.getByName("google.com");

      System.out.println("Host Name: " + ip.getHostName());
      System.out.println("IP Address: " + ip.getHostAddress());

      InetAddress local = InetAddress.getLocalHost();
      System.out.println("Local Host: " + local);

    } catch(Exception e) {
      System.out.println(e);
    }
```

```
    }
}
```

🔹 Applications of InetAddress

- **DNS lookup** (convert domain name to IP)
- **Network communication setup**
- **Checking connectivity** (isReachable)
- **Client-server programming**
- Used in **socket programming**

✍️ Conclusion (Exam Ready)

The InetAddress class is used to represent IP addresses and perform hostname resolution in Java networking. It provides useful methods to retrieve host information and is widely used in network-based applications.

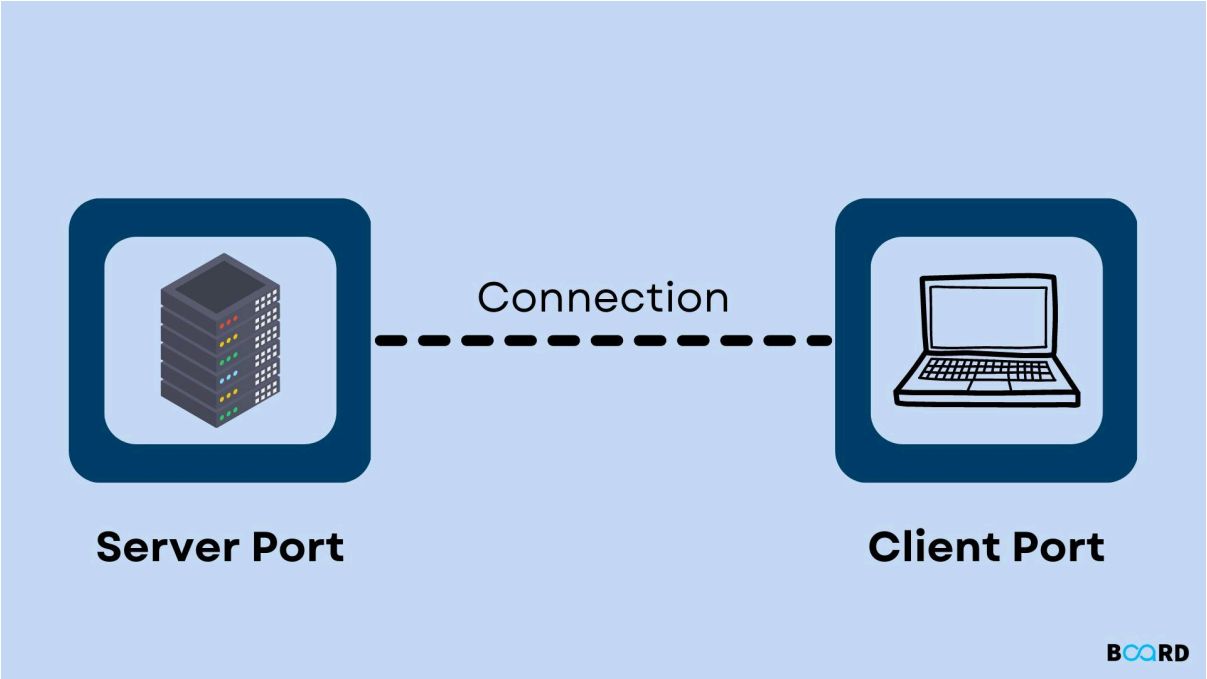
2. Explain TCP/IP Socket programming in java with Client Server Architecture.

TCP/IP Socket Programming in Java (Client–Server Architecture)

Socket programming in Java is used to enable communication between two systems over a network using the **TCP/IP protocol**. It follows a **client–server model**, where one program (server) provides services and another (client) requests them.

(*Covered in your Networking unit – TCP/IP Sockets section*).

📊 Client–Server Architecture



🔹 Key Components

- **Server Socket (ServerSocket)**
  - Waits for client requests
  - Binds to a port number
- **Client Socket (Socket)**
  - Connects to server using IP address and port
- **Streams**
  - InputStream → receive data
  - OutputStream → send data

🔄 Working Steps

🖥️ Server Side

1. Create ServerSocket with port number
2. Wait for client using accept()
3. Establish connection
4. Use streams to communicate
5. Close connection

💻 Client Side

1. Create Socket using server IP and port
2. Establish connection
3. Use streams to send/receive data
4. Close connection

🔹 Example Program

✅ Server Code

```
import java.net.*;
import java.io.*;

class Server {
  public static void main(String[] args) {
    try {
      ServerSocket ss = new ServerSocket(5000);
      System.out.println("Server waiting...");

      Socket s = ss.accept();
      System.out.println("Client connected");

      DataInputStream dis = new DataInputStream(s.getInputStream());
      String msg = dis.readUTF();
      System.out.println("Client says: " + msg);

      ss.close();
    } catch(Exception e) {
      System.out.println(e);
    }
  }
}
```

✅ Client Code

```
import java.net.*;
import java.io.*;

class Client {
  public static void main(String[] args) {
    try {
      Socket s = new Socket("localhost", 5000);
```

```
        DataOutputStream dos = new DataOutputStream(s.getOutputStream());
        dos.writeUTF("Hello Server");

        dos.flush();
        dos.close();
        s.close();
      } catch(Exception e) {
        System.out.println(e);
      }
    }
}
```

🔹 Advantages of TCP/IP Sockets

- Reliable communication (TCP ensures delivery)
- Connection-oriented
- Supports bidirectional communication

✍️ Conclusion (Exam Ready)

TCP/IP socket programming in Java enables communication between client and server using sockets. The server listens on a port, and the client connects to it. Data is exchanged using input and output streams, forming the basis of network applications.

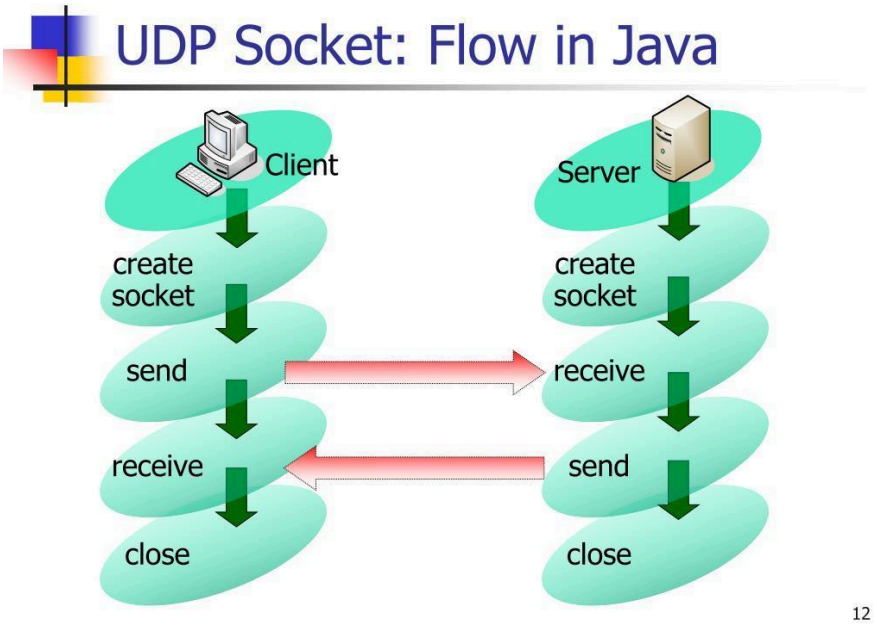
3. Explain the working of datagram packet and datagram socket class.

DatagramPacket and DatagramSocket in Java (UDP Communication)

In Java networking, **DatagramPacket** and **DatagramSocket** classes (from java.net package) are used for **UDP (User Datagram Protocol)** communication.
UDP is **connectionless**, meaning data is sent as independent packets without establishing a connection.

(*Covered in your Networking unit – Datagram Sockets section*).

📊 Working Concept (UDP Communication)



🔹 1. DatagramPacket Class

- Represents a **packet of data** to be sent or received
- Contains:
  - Data (byte array)
  - Length of data
  - Destination IP address
  - Port number

✅ Constructors:

DatagramPacket(byte[] buf, int length)
DatagramPacket(byte[] buf, int length, InetAddress address, int port)

✔️ Important Methods:

- getData() → returns data buffer
- getLength() → returns length of data
- getAddress() → returns sender/receiver IP
- getPort() → returns port number

🔹 2. DatagramSocket Class

- Used to **send and receive DatagramPacket**
- Works without connection (connectionless communication)

✅ Constructors:

DatagramSocket()
DatagramSocket(int port)

✔️ Important Methods:

- send(DatagramPacket p) → sends packet
- receive(DatagramPacket p) → receives packet
- close() → closes socket

🔄 Working Process

📤 Sending Data

1. Create DatagramSocket
2. Create DatagramPacket with data, IP, port
3. Call send()

📥 Receiving Data

1. Create DatagramSocket with port
2. Create empty DatagramPacket
3. Call receive()

🔹 Example

✅ Sender

```
import java.net.*;

class Sender {
  public static void main(String[] args) throws Exception {
    DatagramSocket ds = new DatagramSocket();

    String msg = "Hello";
    byte[] data = msg.getBytes();
```

```
    InetAddress ip = InetAddress.getByName("localhost");

    DatagramPacket dp = new DatagramPacket(data, data.length, ip, 3000);
    ds.send(dp);
    ds.close();
  }
}
```

✅ Receiver

```
import java.net.*;

class Receiver {
  public static void main(String[] args) throws Exception {
    DatagramSocket ds = new DatagramSocket(3000);

    byte[] data = new byte[1024];
    DatagramPacket dp = new DatagramPacket(data, data.length);

    ds.receive(dp);

    String msg = new String(dp.getData(), 0, dp.getLength());
    System.out.println("Received: " + msg);

    ds.close();
  }
}
```

🔍 Key Points

- **Connectionless communication** (no handshake)
- Faster but **less reliable** than TCP
- No guarantee of delivery or order

✍️ Conclusion (Exam Ready)

DatagramPacket represents data packets, and DatagramSocket is used to send and receive them using UDP protocol. This approach enables fast, connectionless communication between systems.

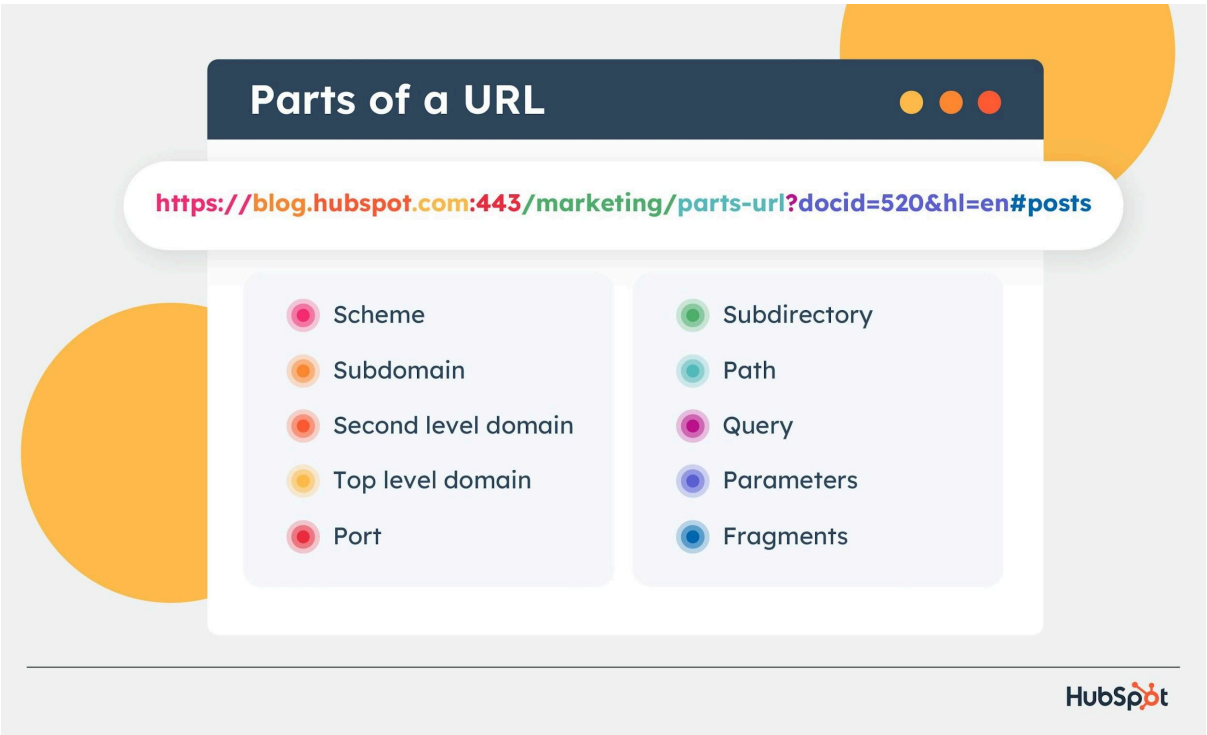
4. Explain URL class in java.

URL Class in Java

The **URL (Uniform Resource Locator) class** in Java is part of the java.net package and is used to represent the **address of a resource on the Internet** (such as a web page, file, or service).

(*Covered in your Networking unit – Java URL class section*).

📊 URL Structure



🔹 Parts of a URL

A URL consists of:

- **Protocol** → e.g., http, https, ftp
- **Host** → domain name (example.com)
- **Port** → optional (default: 80 for HTTP, 443 for HTTPS)
- **File/Path** → resource location (eg. /folder/index.html)

Example:

https://www.example.com:443/index.html

🔹 Creating a URL Object

URL url = new URL("https://www.example.com/index.html");

🔹 Important Methods of URL Class

Method	Description
getProtocol()	Returns protocol (http, https)
getHost()	Returns host name
getPort()	Returns port number
getFile()	Returns file/path
getPath()	Returns path
getQuery()	Returns query string
openStream()	Opens input stream to read data

🔹 Example Program

```
import java.net.*;
import java.io.*;

class Demo {
  public static void main(String[] args) {
    try {
      URL url = new URL("https://www.example.com/index.html");

      System.out.println("Protocol: " + url.getProtocol());
      System.out.println("Host: " + url.getHost());
      System.out.println("Port: " + url.getPort());
      System.out.println("File: " + url.getFile());

    } catch(Exception e) {
      System.out.println(e);
    }
  }
```

}

🔹 Applications

- Access web resources
- Read data from internet
- Used in networking and web applications
- Works with URLConnection for advanced operations

✍️ Conclusion (Exam Ready)

The URL class in Java is used to represent and access resources on the Internet. It provides methods to extract different parts of a URL and to read data from web resources.

Chapter 2- RMI

5. Describe working of RMI With a neat diagram.

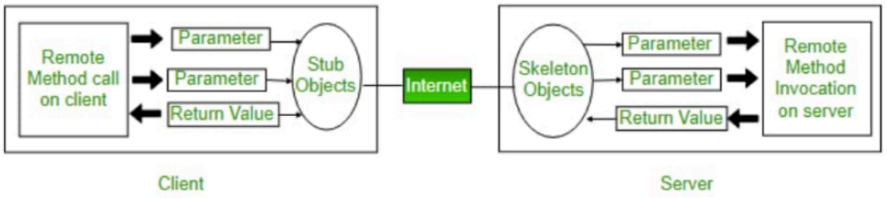
Remote Method Invocation (RMI) – Working

RMI (Remote Method Invocation) is a Java technology that allows an object in one JVM (client) to **invoke methods on an object running in another JVM (server)**, just like calling a local method.

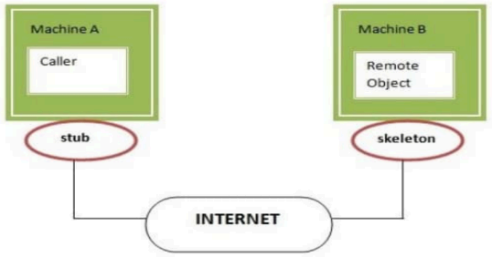
(*Covered in your Unit 4 – RMI section*).

📊 RMI Architecture Diagram

Working of RMI



Working of RMI



🔹 Components of RMI

1. **Remote Interface**
   - Declares methods that can be called remotely
2. **Remote Object (Implementation)**
   - Implements the remote interface
3. **Stub (Client-side proxy)**
   - Acts as a representative of the remote object
4. **Skeleton (Server-side dispatcher)** *(older concept, now handled internally)*
5. **RMI Registry**
   - Stores references of remote objects

🔄 Working of RMI (Step-by-Step)

1. Server creates a **remote object**
2. Server **registers** it with RMI Registry
3. Client looks up the object from Registry
4. Client gets **stub (proxy)**
5. Client calls method on stub
6. Stub sends request to server
7. Server executes method
8. Result is returned to client

🔹 Simple Flow

Client → Stub → Network → Server → Remote Object
← Result Returned ←

🔹 Example (Basic Structure)

Remote Interface

```
import java.rmi.*;

public interface Hello extends Remote {
  String sayHello() throws RemoteException;
}
```

Server Implementation

```
import java.rmi.*;
import java.rmi.server.*;

public class HelloImpl extends UnicastRemoteObject implements Hello {
  HelloImpl() throws RemoteException {}

  public String sayHello() {
    return "Hello from Server";
  }
}
```

🔹 Advantages

- Simplifies distributed computing
- Allows remote method calls like local calls
- Supports object-oriented programming

✍️ Conclusion (Exam Ready)

RMI enables communication between distributed Java applications by allowing a client to invoke methods on a remote object via stubs and registry. It simplifies network programming using object-oriented concepts.

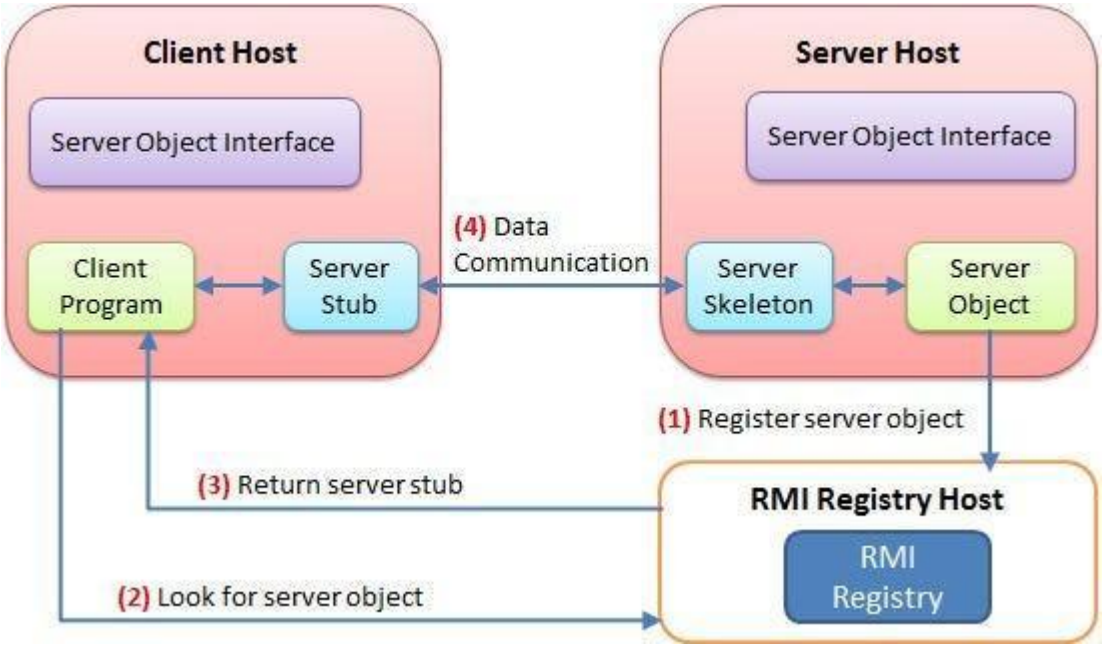
6. Explain RMI Registry.

RMI Registry in Java

The **RMI Registry** is a **naming service** in Java RMI that allows clients to **locate (lookup) remote objects** available on a server. It acts like a **directory or phone book** where remote objects are registered with a name.

(*Covered in your Unit 4 – RMI working section*).

📊 RMI Registry Working



🔹 Key Functions of RMI Registry

- **Register (bind)** remote objects with a name
- **Lookup** remote objects by clients
- Acts as a **middle layer** between client and server

🔹 Important Methods (from java.rmi.Naming class)

Method	Description
bind(name, object)	Registers a new remote object
rebind(name, object)	Replaces existing binding
lookup(name)	Retrieves remote object
unbind(name)	Removes binding

🔹 Working Process

1. Server creates remote object
2. Server registers it with RMI Registry using bind() or rebind()
3. Client calls lookup() using object name
4. Registry returns reference (stub) to client
5. Client invokes methods on remote object

🔹 Example

Server Side

```
import java.rmi.*;
import java.rmi.registry.*;

public class Server {
  public static void main(String[] args) throws Exception {
    HelloImpl obj = new HelloImpl();

    LocateRegistry.createRegistry(1099); // default port
    Naming.rebind("HelloService", obj);

    System.out.println("Server ready...");
  }
}
```

Client Side

```
import java.rmi.*;
```

```
public class Client {
  public static void main(String[] args) throws Exception {
    Hello h = (Hello) Naming.lookup("rmi://localhost/HelloService");
    System.out.println(h.sayHello());
  }
}
```

🔹 Key Points

- Default port number: **1099**
- Must be running before client lookup
- Stores only **references**, not actual objects

✍️ Conclusion (Exam Ready)

The RMI Registry is a naming service that allows remote objects to be registered and located using names. It enables communication between client and server in distributed applications.

Chapter 3 - JDBC

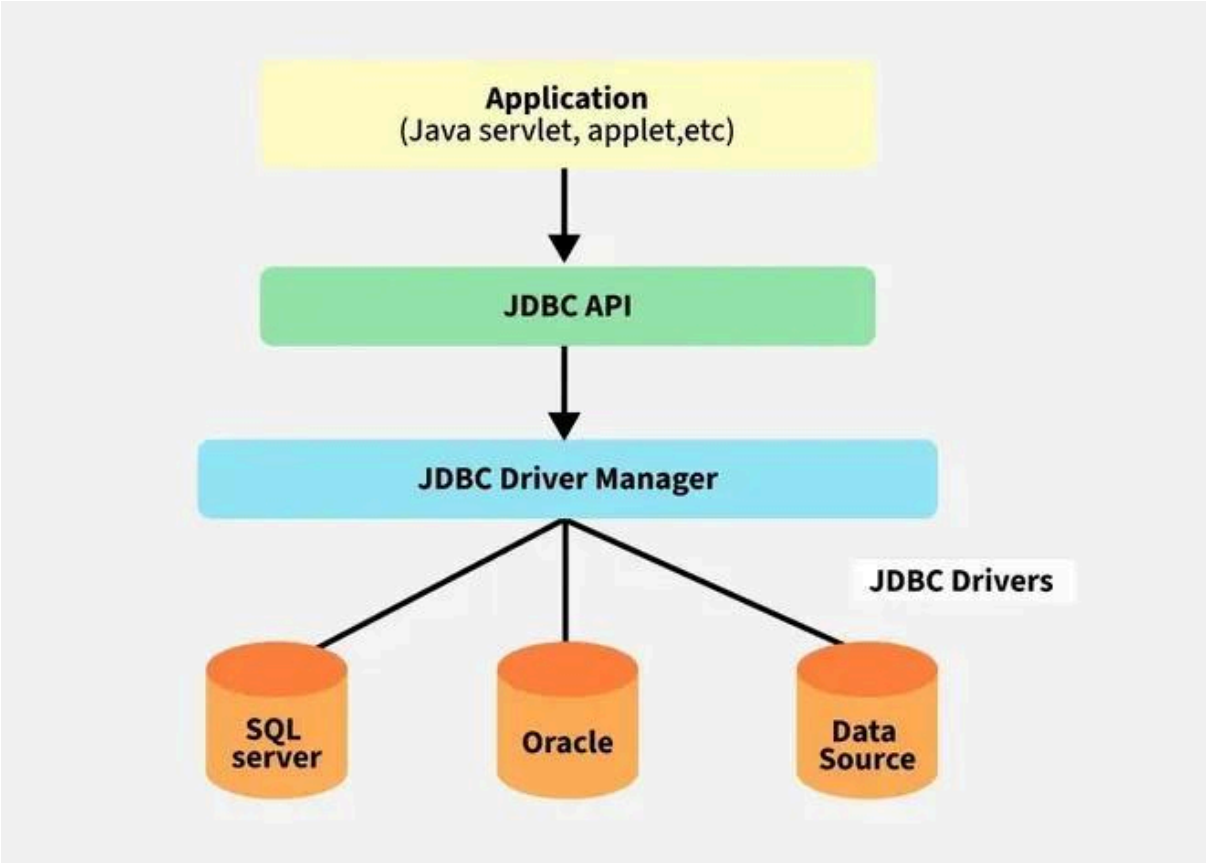
7.Explain the Architecture of JDBC and types of JDBC Driver.

JDBC Architecture and Types of JDBC Drivers

JDBC (Java Database Connectivity) is an API used to connect Java applications with databases. It provides a standard way to **execute SQL queries and interact with databases**.

(*Covered in your Unit 4 – JDBC section*).

📊 JDBC Architecture



🔹 Components of JDBC Architecture

1. **Java Application**
   - User program that interacts with database
2. **JDBC API**
   - Interfaces like Connection, Statement, ResultSet
3. **DriverManager**
   - Manages database drivers and connections
4. **JDBC Driver**
   - Converts Java calls into database-specific calls
5. **Database**
   - Stores actual data

🔄 Working Flow

1. Load driver
2. Establish connection (Connection)
3. Create statement (Statement / PreparedStatement)

4. Execute query
5. Process result (ResultSet)
6. Close connection

🔹 Types of JDBC Drivers

1️⃣ Type 1: JDBC-ODBC Bridge Driver

- Uses ODBC driver to connect database
- Platform dependent
- **Deprecated**

2️⃣ Type 2: Native-API Driver

- Uses database-specific native libraries
- Faster than Type 1
- Requires installation on client

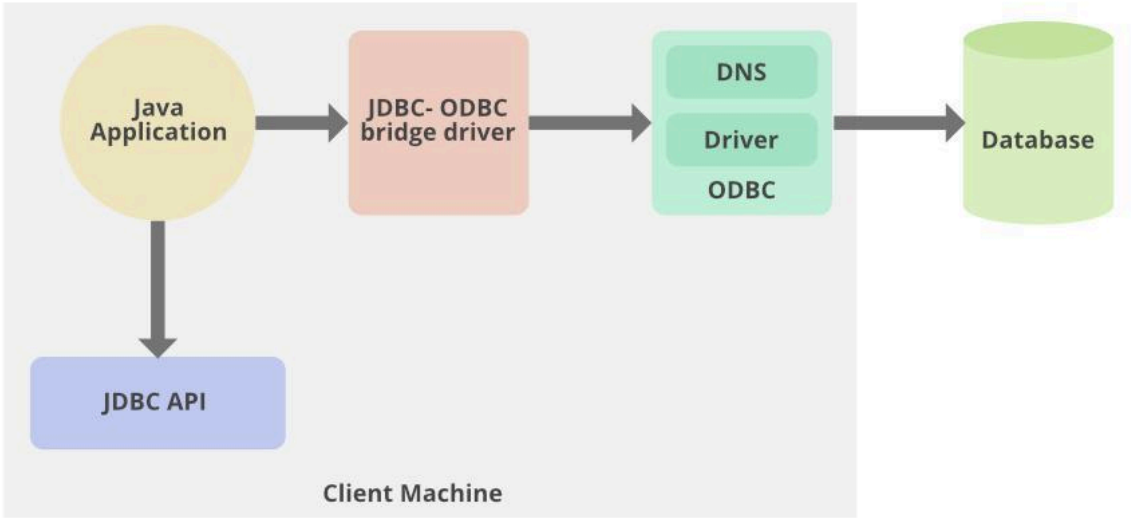
3️⃣ Type 3: Network Protocol Driver

- Uses middleware server
- Sends requests to database via network
- Platform independent

4️⃣ Type 4: Thin Driver (Pure Java)

- Direct communication with database
- No native libraries required
- Most commonly used

📊 Driver Types Overview



🔍 Comparison Summary

Type	Speed	Platform Dependency	Usage
Type 1	Slow	Yes	Deprecated
Type 2	Medium	Yes	Limited use
Type 3	Medium	No	Rare
Type 4	Fast	No	Most used

✍️ Conclusion (Exam Ready)

JDBC architecture consists of components like DriverManager, Connection, Statement, and ResultSet that enable database interaction. There are four types of JDBC drivers, among which the Type 4 driver is the most efficient and widely used.

8. Differentiate between Statement, Prepared Statement and Callable Statement.

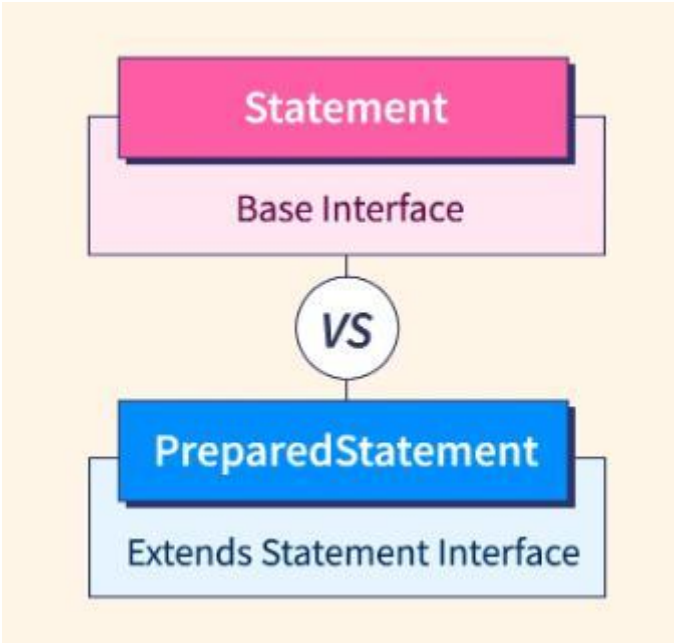
Difference between Statement, PreparedStatement and CallableStatement

In JDBC, these three interfaces are used to execute SQL queries, but they differ in **performance, flexibility, and usage**.

📊 Concept Overview

Different Types of Statements

- **Overview**
  - Through the Statement object, SQL statements are sent to the database.
  - Three types of statement objects are available:
    1. **Statement**
       - for executing a simple SQL statements
    2. **PreparedStatement**
       - for executing a precompiled SQL statement passing in parameters
    3. **CallableStatement**
       - for executing a database stored procedure



🔍 Differences

Feature	Statement	PreparedStatement	CallableStatement
Query Type	Simple SQL queries	Precompiled SQL with parameters	Stored procedures
Compilation	Compiled every time	Compiled once	Precompiled
Parameters	Not supported	Supported (?)	Supported (IN, OUT, INOUT)
Performance	Slow	Faster	Faster
Security	Prone to SQL Injection	Prevents SQL Injection	Secure
Usage	Static queries	Dynamic queries	Database procedures

🔹 1. Statement

- Used for **simple SQL queries**
- Query is compiled every time

✅ Example:

```
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("SELECT * FROM users");
```

🔹 2. PreparedStatement

- Used for **parameterized queries**
- Precompiled → improves performance

✅ Example:

```
PreparedStatement ps = con.prepareStatement(
  "SELECT * FROM users WHERE id = ?"
```

);
ps.setInt(1, 101);
ResultSet rs = ps.executeQuery();

🔹 3. CallableStatement

- Used to call **stored procedures**
- Supports IN, OUT parameters

✅ **Example:**

CallableStatement cs = con.prepareCall("{call getUser(?)}");
cs.setInt(1, 101);
ResultSet rs = cs.executeQuery();

🔹 Key Points

- PreparedStatement is **most commonly used**
- CallableStatement is used when working with **database procedures**
- Statement is rarely preferred due to **performance and security issues**

✍️ Conclusion (Exam Ready)

Statement, PreparedStatement, and CallableStatement are JDBC interfaces used to execute SQL queries. PreparedStatement is faster and more secure than Statement, while CallableStatement is used to execute stored procedures.